

1. Projekttitlel

Konzeption und Entwicklung einer webbasierten Multiplayer-Plattform für Echtzeit-Strategiespiele im lokalen Netzwerk am Beispiel von 'Schiffe versenken'

2. Ausbildungsberuf

Fachinformatiker für Anwendungsentwicklung

3. Auftraggeber

Schulprojekt – Eigeninitiative

4. Projektart

Das Projekt wird als Einzelprojekt durchgeführt.

Der Schwerpunkt liegt auf der Konzeption und Entwicklung einer webbasierten Plattform, über die zwei Spieler das klassische Spiel 'Schiffe versenken' in Echtzeit im lokalen Netzwerk gegeneinander spielen können. Das Projekt zeigt exemplarisch, wie eine solche Multiplayer-Architektur für rundenbasierte Strategiespiele in Python umgesetzt werden kann.

5. Ausgangssituation

Das klassische Brettspiel 'Schiffe versenken' erfordert die physische Anwesenheit beider Spieler sowie entsprechendes Spielmaterial. Eine digitale Umsetzung über den Browser würde es ermöglichen, das Spiel bequem von verschiedenen Endgeräten aus zu spielen, ohne zusätzliche Software installieren zu müssen.

Aktuell existiert keine einfache, leichtgewichtige und browserbasierte Multiplayer-Lösung für dieses Spiel, die ohne externe Serverinfrastruktur oder Internetverbindung auskommt und dennoch eine flüssige Echtzeit-Kommunikation zwischen den Spielern ermöglicht.

Ziel ist es daher, eine Softwarelösung zu entwickeln, mit der zwei Spieler 'Schiffe versenken' vollständig über den Browser im lokalen Netzwerk spielen können.

6. Projektziel

Ziel des Projekts ist die Entwicklung einer webbasierten Multiplayer-Plattform, die das klassische Spiel 'Schiffe versenken' als Echtzeit-Strategiespiel im lokalen Netzwerk umsetzt. Die Anwendung soll vollständig in Python entwickelt werden und ausschließlich über einen Standard-Webbrowser erreichbar sein, ohne dass zusätzliche Software auf den Endgeräten der Spieler installiert werden muss.

Beide Spieler sollen sich über einen gemeinsamen Raumcode verbinden, ihre Schiffe auf einem interaktiven 10x10-Spielfeld platzieren und anschließend abwechselnd ihre Züge ausführen. Treffer und Versenkungen sollen in Echtzeit auf beiden Bildschirmen angezeigt werden. Das Spiel endet automatisch, sobald alle Schiffe eines Spielers versenkt wurden.

Das Projekt soll eine stabile, benutzerfreundliche und testbare Lösung liefern, die über WLAN erreichbar ist und auf einem normalen Rechner im lokalen Netzwerk ausgeführt werden kann.

7. Projektumfang

Im Rahmen des Projekts werden folgende Hauptfunktionen umgesetzt:

Muss-Funktionen

- Lobby-System: Erstellen und Beitreten von Spielräumen über einen Raumcode im Browser
- Spielfeld-Darstellung: Interaktives 10x10-Gitter für Schiffsplatzierung und Spielverlauf
- Schiffsplatzierung: Spieler platzieren ihre Schiffe vor Spielbeginn per Klick auf das Spielfeld
- Echtzeit-Kommunikation: Spielzüge werden sofort per WebSocket an den Gegner übertragen
- Spiellogik (serverseitig): Trefferauswertung, Versenkung von Schiffen, Erkennung der Siegbedingung
- Spielstatus-Anzeige: Anzeige des aktuellen Spielers, Treffer- und Wasser-Markierungen sowie Spielende-Meldung
- Verbindung über lokales WLAN ohne Internetzugang
- Entwicklung der gesamten Spiellogik in Python
- Testspiel mit dokumentierten Ergebnissen

Kann-Funktionen

- Chat-Funktion: Textnachrichten zwischen den Spielern während der Partie
- Zuschauer-Modus: Dritte Personen können eine laufende Partie beobachten
- Spielstatistik: Anzeige von Anzahl der Züge sowie Treffer- und Fehlschuss-Quote je Spieler
- Mehrere gleichzeitige Spielräume auf demselben Server
- Benutzeranleitung zum Starten und Bedienen der Anwendung

8. Projektabgrenzung

Nicht Bestandteil des Projekts ist ein Einzelspielermodus mit KI-Gegner. Das Spiel ist ausschließlich für zwei menschliche Spieler im lokalen Netzwerk konzipiert.

Ebenfalls nicht Bestandteil des Projekts sind eine native App für Smartphones, ein Betrieb über das Internet, eine Datenbankspeicherung von Spielverläufen oder Nutzerkonten sowie eine produktive Sicherheitszertifizierung.

Der Schwerpunkt des Projekts liegt auf der Softwareentwicklung, der browserbasierten Spieloberfläche, der Echtzeit-Kommunikation per WebSocket und der serverseitigen Spiellogik in Python.

9. Technische Rahmenbedingungen

Hardware

- Rechner oder Laptop als Server im lokalen Netzwerk
- Endgeräte der Spieler mit aktuellem Webbrowser (Chrome, Firefox oder Edge)

Software und Werkzeuge

- Programmiersprache: Python 3.x
- Backend-Framework: Flask mit Flask-SocketIO für WebSocket-Echtzeit-Kommunikation
- Frontend: HTML5, CSS3, JavaScript (ES6) ohne externe Frameworks

- Entwicklungsumgebung: Visual Studio Code oder PyCharm
- Versionsverwaltung: Git-Repository
- Netzwerkzugriff: lokales WLAN, kein Internetzugang erforderlich

10. Geplante Umsetzung

Zunächst werden die Anforderungen an das Spiel analysiert und die Spielregeln sowie das Datenmodell festgelegt. Anschließend wird die Softwarearchitektur geplant. Dabei wird festgelegt, wie Browser, Flask-Server und WebSocket-Kommunikation zusammenspielen und wie die Spiellogik serverseitig abgebildet wird.

Danach wird die Entwicklungsumgebung eingerichtet und die Grundstruktur des Python-Projekts erstellt. Im nächsten Schritt wird das Lobby-System entwickelt, über das Spieler Räume erstellen und beitreten können.

Anschließend wird das Spielfeld im Browser umgesetzt. Dazu gehören die Darstellung des 10x10-Gitters, die Schiffsplatzierung durch den Spieler sowie die Markierung von Treffern und Fehlschüssen im Spielverlauf.

Parallel dazu wird die Spiellogik auf dem Server implementiert. Der Server verwaltet den Spielzustand, wertet Züge aus, erkennt versenkte Schiffe und stellt den Sieg fest. Die Echtzeit-Übertragung der Züge an beide Spieler erfolgt per WebSocket.

Zum Abschluss werden Tests durchgeführt. Dabei wird geprüft, ob die Verbindung zwischen beiden Spielern stabil ist, die Spiellogik korrekt funktioniert und das Spiel fehlerfrei zu Ende gespielt werden kann. Die Ergebnisse werden in einem Testprotokoll dokumentiert.

11. Zeitplanung

Projektphase	Geplante Zeit
Analyse der Anforderungen und Spielregeln	5 Stunden
Planung der Softwarearchitektur und Datenmodell	7 Stunden
Einrichtung der Entwicklungsumgebung und Projektstruktur	5 Stunden
Entwicklung des Lobby-Systems (Raumverwaltung)	10 Stunden
Entwicklung der Spieloberfläche im Browser (Frontend)	12 Stunden
Entwicklung der Spiellogik (serverseitig, Python)	13 Stunden
Integration der WebSocket-Kommunikation	8 Stunden
Tests und Fehlerbehebung	10 Stunden
Projektdokumentation	10 Stunden
Gesamt	80 Stunden

11. Kostenkalkulation (Wirtschaftlichkeitsbetrachtung)

Da es sich bei diesem Projekt um ein Schulprojekt im Rahmen einer Umschulung handelt, werden die Personalkosten auf Basis eines fiktiven, aber marktüblichen Stundensatzes für Umschüler berechnet. Für die Sachkosten wird die vorhandene Infrastruktur genutzt, wobei die anteiligen Stromkosten für die Projektlaufzeit detailliert berücksichtigt werden.

Berechnung der Stromkosten:

- Leistungsaufnahme (Laptop): ca. 45 Watt (0,045 kW)
- Strompreis (Berlin): ca. 0,30 € / kWh
- Projektdauer: 80 Stunden

$$\text{Stromkosten} = 0,045 \text{ kW} \times 80 \text{ Std.} \times 0,30 \text{ euro/kWh} = 1,08 \text{ euro}$$

Kostentabelle (Gesamtkostenübersicht):

Kostenart	Beschreibung	Menge	Einzelprei s	Gesamtprei s
Personalkosten	Umschüler (Eigenleistung)	80 Std.	13,00 € / Std.	1.040,00 €
Hardwarekosten	Laptop / Entwicklungsrechner	vorhanden	0,00 €	0,00 €
Softwarekosten	Python, Flask, VS Code	kostenlos (Open Source)	0,00 €	0,00 €
Stromkosten	Betrieb des Laptops (45W)	80 Std.	0,014 € / Std.	1,08 €
Netzwerkkosten	WLAN-Router für lokales Netz	vorhanden	0,00 €	0,00 €
Gesamtkosten				1.041,08 €

12. Projektphasen im Detail

1. Analyse der Anforderungen und Spielregeln – 5 Stunden
 - Festlegung der genauen Spielregeln und Spielfeldgröße
 - Definition der Muss- und Kann-Funktionen

- Analyse der technischen Möglichkeiten von Flask und Socket.io
 - Festlegung der Testumgebung
2. Planung der Softwarearchitektur und Datenmodell – 7 Stunden
 - Planung der Kommunikation zwischen Browser und Server per WebSocket
 - Entwurf des Datenmodells: Spielfeld, Schiffe, Spielzustand
 - Definition aller WebSocket-Events (z. B. join_room, place_ship, fire, game_over)
 - Strukturierung des Python-Projekts
 - Erstellung von Wireframes für die Spieloberfläche
 3. Einrichtung der Entwicklungsumgebung und Projektstruktur – 5 Stunden
 - Installation und Konfiguration von Python, Flask, Flask-SocketIO
 - Einrichtung des Git-Repositorys
 - Erstellung der grundlegenden Projektstruktur (Ordner, Dateien, Templates)
 4. Entwicklung des Lobby-Systems – 10 Stunden
 - Umsetzung der Raumerstellung mit eindeutigem Raumcode
 - Umsetzung des Raumbetrtritts über den Raumcode
 - Verwaltung der Spieler pro Raum auf dem Server
 - Weiterleitung beider Spieler zum Spielfeld nach erfolgreichem Beitritt
 5. Entwicklung der Spieloberfläche im Browser – 12 Stunden
 - Umsetzung des 10x10-Spielfelds als interaktives HTML/CSS-Gitter
 - Umsetzung der Schiffsplatzierung per Klick
 - Anzeige von Treffer- und Wasser-Markierungen im Spielverlauf
 - Anzeige des aktuellen Spielers und der Spielende-Meldung
 - Responsives Layout für verschiedene Bildschirmgrößen
 6. Entwicklung der Spiellogik – 13 Stunden
 - Implementierung der Trefferauswertung auf dem Server
 - Erkennung von versenkten Schiffen
 - Implementierung der Zugwechsel-Logik
 - Erkennung der Siegbedingung und Benachrichtigung beider Spieler
 - Grundlegende Fehlerbehandlung bei ungültigen Zügen
 7. Integration der WebSocket-Kommunikation – 8 Stunden
 - Verknüpfung aller Frontend-Aktionen mit den entsprechenden WebSocket-Events
 - Sicherstellung der Echtzeit-Übertragung an beide Spieler
 - Test der Verbindungsstabilität im lokalen Netzwerk
 8. Tests und Fehlerbehebung – 10 Stunden
 - Funktionstest des Lobby-Systems
 - Test der Schiffsplatzierung und Spiellogik
 - Vollständiges Testspiel von Anfang bis Ende
 - Fehleranalyse und Fehlerbehebung
 - Erstellung eines Testprotokolls
 9. Projektdokumentation – 10 Stunden
 - Erstellung der Projektdokumentation
 - Beschreibung der Ausgangssituation und Zielsetzung
 - Beschreibung der technischen Umsetzung und Architektur
 - Dokumentation der Tests
 - Erstellung einer kurzen Benutzeranleitung
 - Dokumentation des Quellcodes und des Git-Repositorys

13. Erwartetes Projektergebnis

Am Ende des Projekts soll eine funktionsfähige webbasierte Multiplayer-Plattform vorliegen, über die zwei Spieler 'Schiffe versenken' vollständig im Browser spielen können. Beide Spieler verbinden sich über einen Raumcode, platzieren ihre Schiffe und führen ihre Züge abwechselnd aus. Treffer und Versenkungen werden in Echtzeit auf beiden Bildschirmen angezeigt. Das Spiel endet automatisch mit einer Siegmeldung.

Als Projektergebnis werden abgegeben:

- lauffähige Python-Software (Flask-Server mit Spiellogik)
- Weboberfläche zur Spieldarstellung und Steuerung im Browser
- Quellcode im Git-Repository
- Projektdokumentation
- Testprotokoll mit Ergebnissen des Testspiels
- kurze Benutzeranleitung zum Starten und Bedienen der Anwendung

14. Nutzen des Projekts

Das Projekt zeigt, wie eine browserbasierte Echtzeit-Multiplayer-Plattform mit Python und WebSockets aufgebaut werden kann. Die entwickelte Architektur lässt sich als Grundlage für weitere rundenbasierte oder Echtzeit-Strategiespiele verwenden.

Durch die rein browserbasierte Umsetzung ohne zusätzliche Software-Installation können die Spieler sofort losspielen. Die Anwendung ist leichtgewichtig, einfach zu starten und ausschließlich im lokalen Netzwerk betrieben – ohne Abhängigkeit von externen Diensten oder Internetverbindung.

15. Abgabetermin

Das Projekt muss spätestens am 18.09.2026 abgegeben werden.